TP N° 4
ASP.NET MVC : Contrôleur, Vue, Routage

Rappel - Pour configurer ASP.Net :

Dans Visual Studio Installer, cochez charge de travail :

- ✓ Développement web et ASP.NET
- ✓ Développement .NET Desktop

Composant individuels, ajoutez :

- ✓ ASP .NET MVC4
- ✓ Fonctionnalités ASP.NET avancées
- ✓ Modèles d'éléments et de projet.NET Framework
- ✓ Outils de développement .NET Framework 3.5

1. Créer le projet PremiereAppWeb (via Visual Studio)

- Ouvrir Visual Studio, puis **Fichier > Nouveau > Projet**.
- Choisir **Application web ASP.NET (.NET Framework)** puis **Suivant**.
- Nom du projet : PremiereAppWeb.
- Choisir le modèle **MVC** (cocher MVC) puis **Créer**.

Créer une application permet :

- L'affichage de la **liste des étudiants** ;
- L'affichage de la **liste des professeurs** ;
- L'affichage des **détails d'un étudiant et un professeur** à partir de son identifiant.

2. Créer les modèles

- Dans le dossier Models, ajouter deux classes (clic droit > Ajouter > Classe).
- **Classe Etudiant, fichier : Etudiant.cs**

```
public class Etudiant
{
    public int Id { get; set; }
    public string Nom { get; set; }
    public string Prenom { get; set; }
    public int Age { get; set; }

}
```

- **Classe Prof, fichier : Prof.cs**

```
public class Prof
{
        public int Id { get; set; }
        public string Nom { get; set; }
        public string Matiere { get; set; }
        public int Experience { get; set; }

    }
```

3. Créer les contrôleurs

- ✓ Clic droit sur le dossier Controllers > **Ajouter** > **Contrôleur**
- ✓ Choisir **Contrôleur MVC – Vide**
- ✓ Nom : *EtudiantController.cs*

3.1 Créez une liste statique des étudiants, exemple :

```
        private static readonly List<Etudiant> liste = new List<Etudiant>
{
            new Etudiant { Id = 1, Nom = "Benali", Prenom = "Amine", Age = 21 },
            new Etudiant { Id = 2, Nom = "Nouri", Prenom = "Salma", Age = 22 },
            new Etudiant { Id = 3, Nom = "Zerouali", Prenom = "Said", Age = 20 }
};
```

3.2 Ajoutez l'action Index dans le contrôleur :

```
public ActionResult Index()
{
    return View(liste);
}
```

3.3 Ajouter le fichier ProfController.cs de même logique et refaire les mêmes étapes

Élément	Rôle
ActionResult	Type de retour MVC, permet de retourner une vue, un message, ou une redirection
Index()	Action qui retourne la liste des données
View(...)	Appelle une vue et lui passe un modèle

4. Créer la vue html « Index.cshtml »

Chemins:

- Views/Etudiant/Index.cshtml
- Views/Prof/Index.cshtml

4.1 Ajouter la vue : Views/Etudiant/Index.cshtml :

```
@model List<PremiereAppWeb.Models.Etudiant>
@{
    ViewBag.Title = "Liste des étudiants";
}
<h2>@ViewBag.Title</h2>
<table border="1" cellpadding="5">
    <thead>
        <tr>
            <th>ID</th>
            <th>Nom</th>
            <th>Prénom</th>
            <th>Âge</th>
            <th>Détail</th>
```

```
        </tr>
    </thead>
    <tbody>
        @foreach (var etudiant in Model)
        {
            <tr>
                <td>@etudiant.Id</td>
                <td>@etudiant.Nom</td>
                <td>@etudiant.Prenom</td>
                <td>@etudiant.Age</td>
                <td>@Html.ActionLink("Détail", "Details", new {id = etudiant.Id
})</td>
            </tr>
        }
    </tbody>
</table>
```

NB : `Html.ActionLink(texte, action, valeurs)` **construit l'URL à partir du routage : si la route change, le lien reste correct.**

Élément Razor	Signification
`@model`	Déclare le type du modèle utilisé dans la vue
`@Model.Propriété`	Accès aux données passées depuis le contrôleur
`@foreach`	Boucle sur une liste pour afficher plusieurs lignes
`@{}`	Bloc de code C# intégré dans HTML
`cshtml`	Fichiers Razor : mélangent HTML + C#

4.2 Ajouter la vue : Views/Prof/Index.cshtml de même logique

4.3 Ajouter l'action Details et sa vue pour afficher les détails d'un étudiant (par id)

```
public ActionResult Details(int id)
            {
/*
             * Ajoutez votre code ici
             *
             */
            }
```

4.4 Ajouter l'action Details et sa vue pour afficher les détails d'un professeur (par id)

5. Ajouter les routages

Ouvrir **App_Start/RouteConfig.cs**. Ajouter l'alias **ProfsAlias avant** la route **Default**, et changer le contrôleur par défaut en **Etudiant** :

```
routes.MapRoute(
    name: "ProfsAlias",
    url: "nos-profs/{action}/{id}",
    defaults: new{ controller = "Prof", action = "Index",  id = UrlParameter.Optional
    });
routes.MapRoute(
name: "Default",
url: "{controller}/{action}/{id}",
defaults: new { controller = "Etudiant", action = "Index", id = UrlParameter.Optional
});
```

Élément	Rôle
`routes.MapRoute(...)`	Associe une **URL** à un **contrôleur** et une **action**.
`url: "{controller}/{action}/{id}"`	Modèle de l'URL
`defaults:`	Valeurs par défaut si rien n'est précisé
`UrlParameter.Optional`	Rend le paramètre id **facultatif**.
Routage multiple	Plusieurs routes, testées **dans l'ordre,** la plus spécifique en premier.

6. Exécuter le projet et tester ces URL :

- `/` : liste des étudiants
- `/Etudiant/Details/2` : détail de l'étudiant
- `/Etudiant/Details/90` : erreur 404
- `/Prof` : liste des professeurs
- `/nos-profs` : liste des professeurs, via l'alias.